

第8课 架构模式- 模块拆分模式

汇报人：北京交通大学软件学院

严禁复制

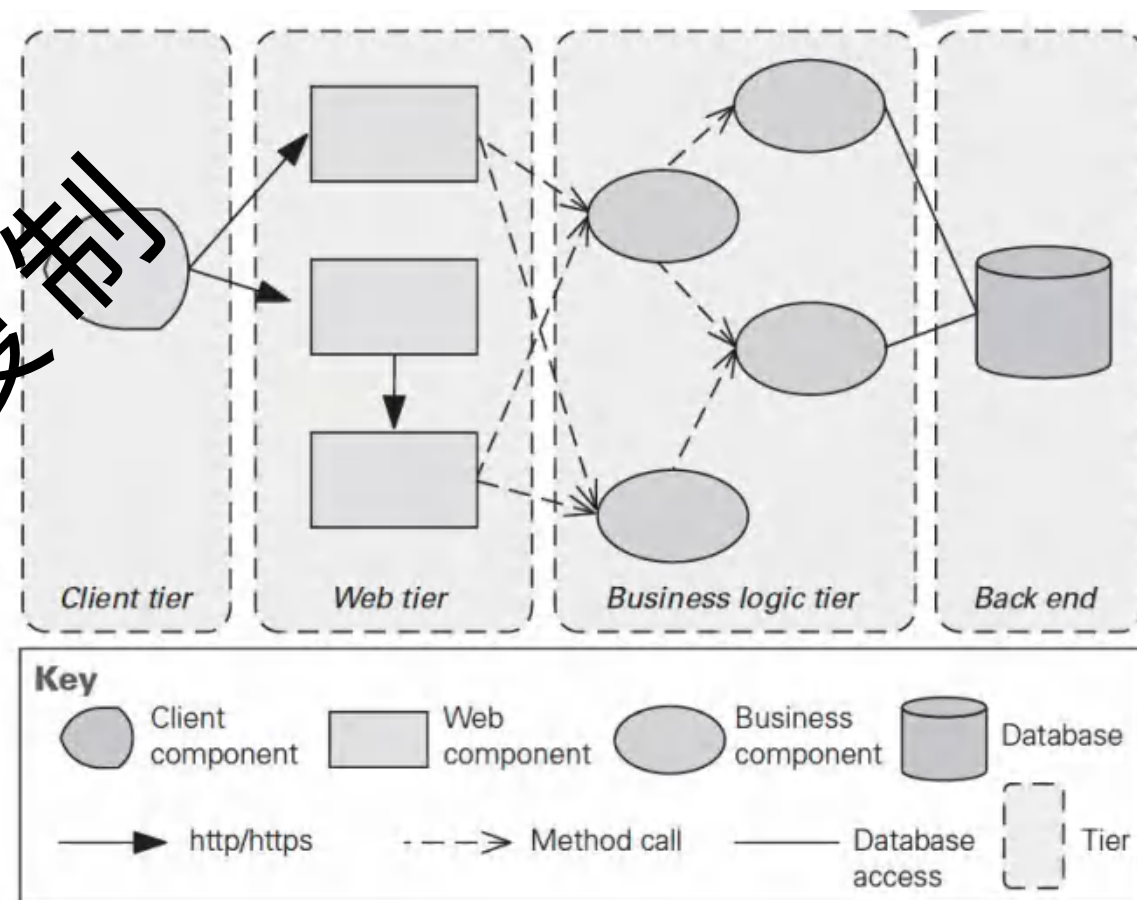
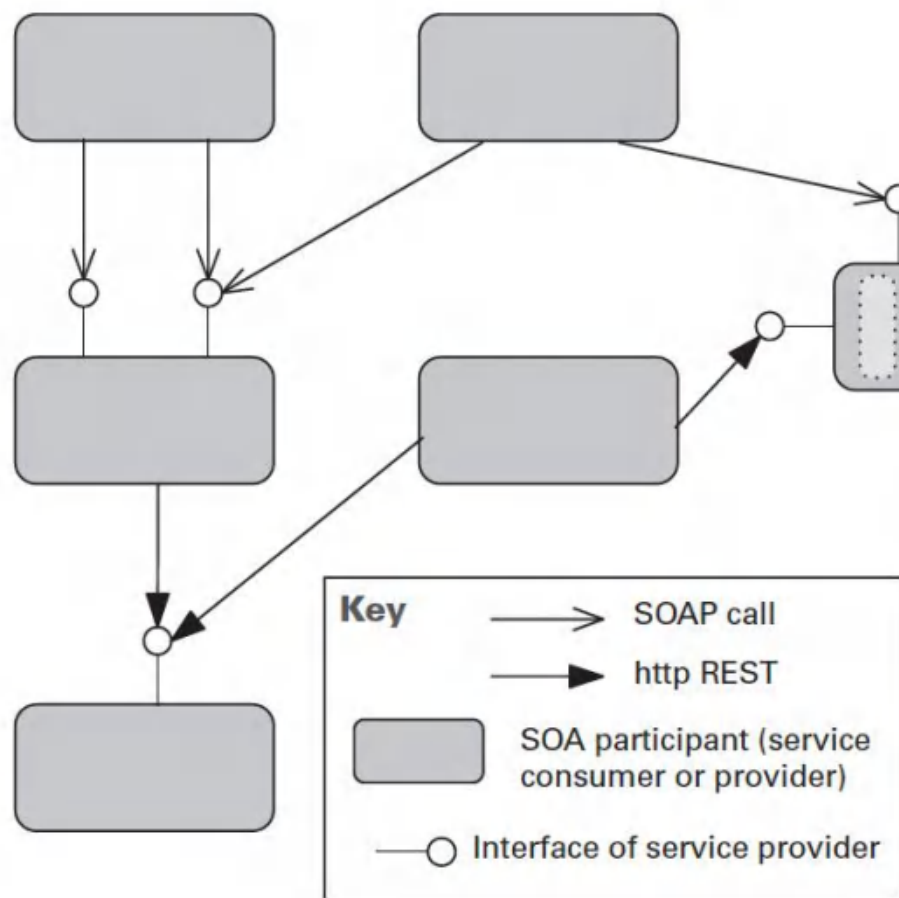


架构模式

- 在一个给定的上下文中，针对一些反复发生的问题，存在可复用的通用解决方案
 - 重要参数：问题
 - 有的文档也称为style，看起来没有区别
- 3种类型
 - 结构模式：构建期间
 - 分层模式
 - 运行时模式（行为）
 - Pipe & filters模式
 - 部署模式
 - load balancer

严禁复制

一个应用可以同时使用多种pattern



Pattern语言和Pattern目录

Pattern目录

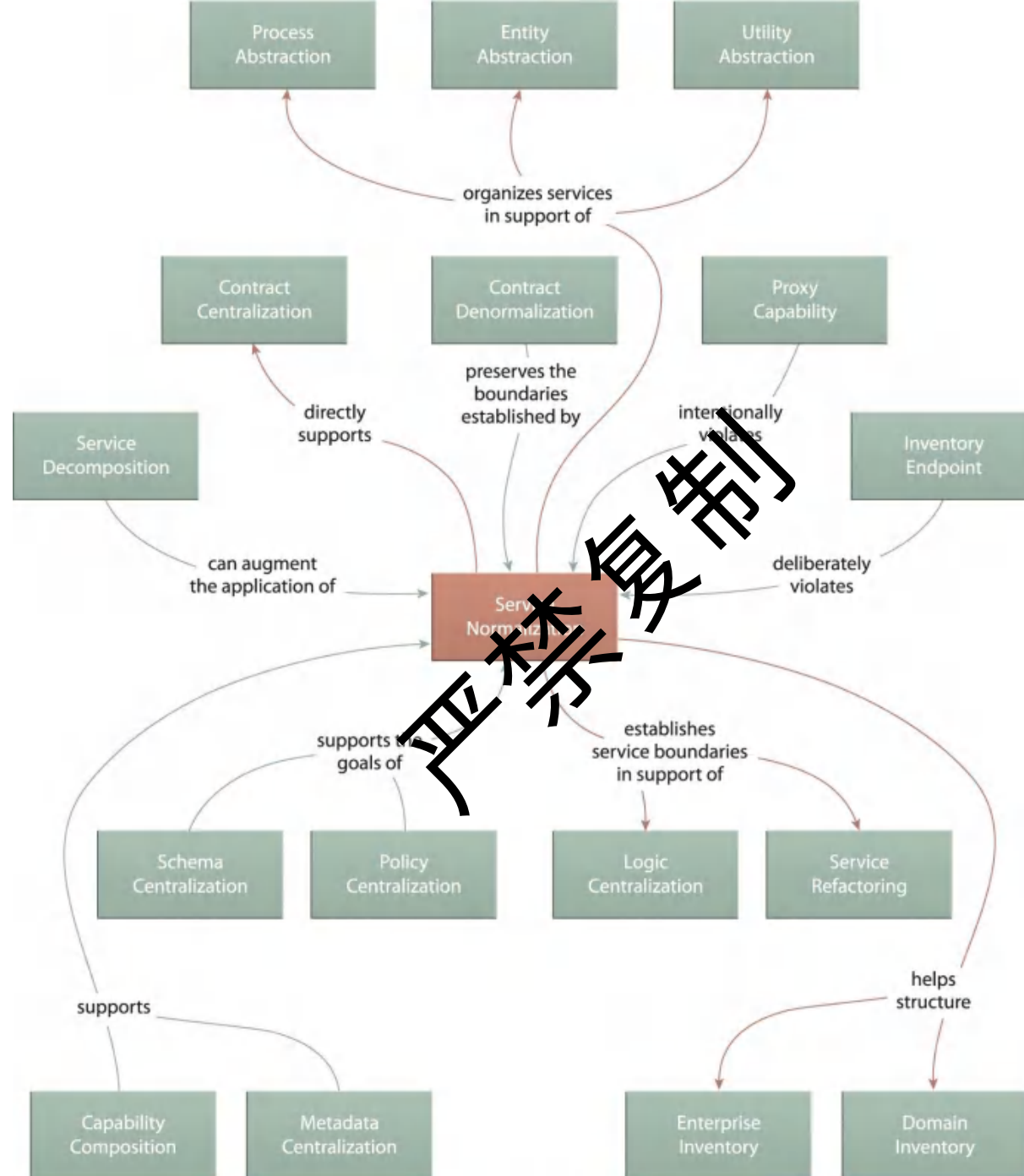
1. 关于一个主题的一系列的pattern;
2. 不一定是完备的;

01

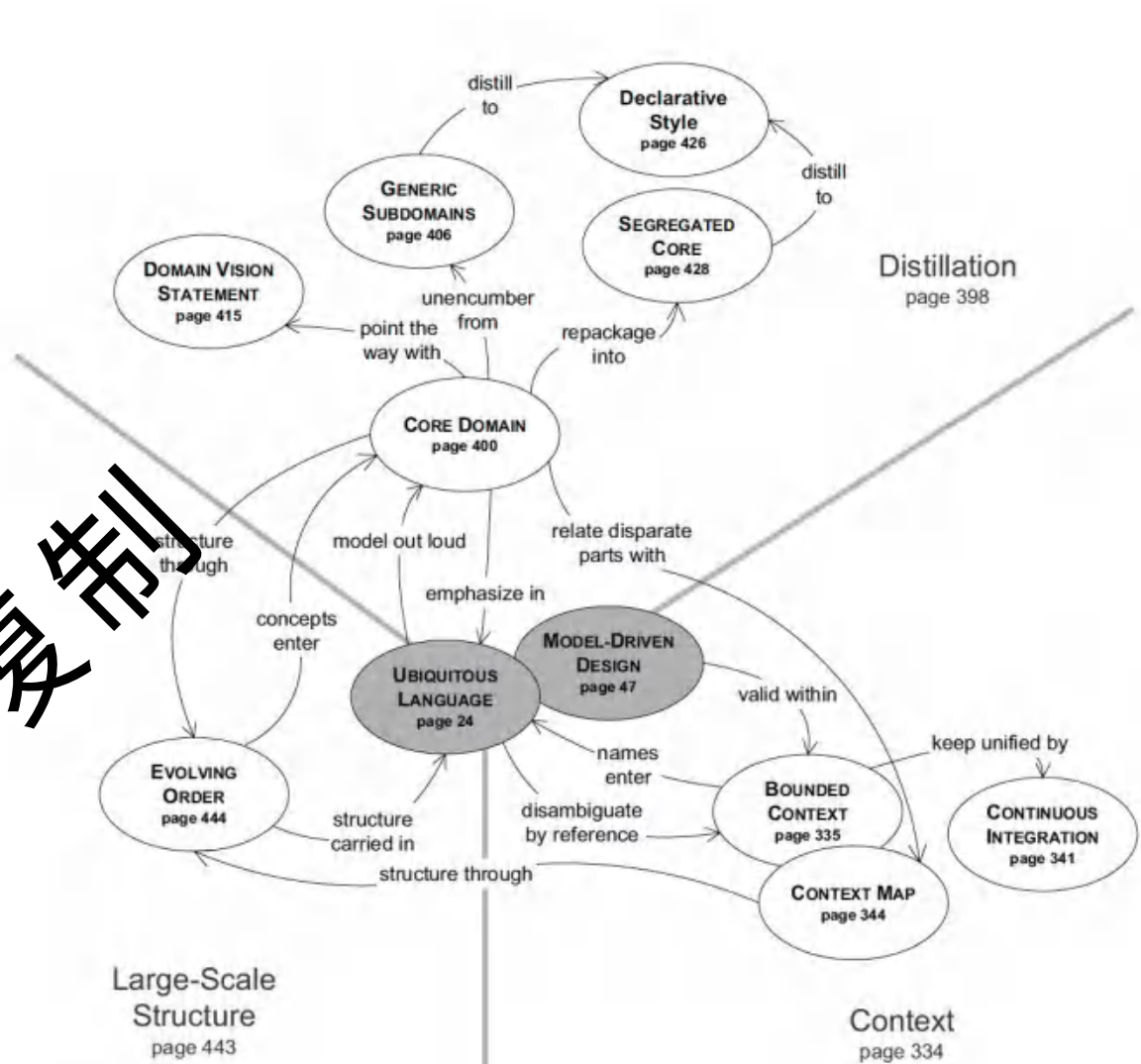
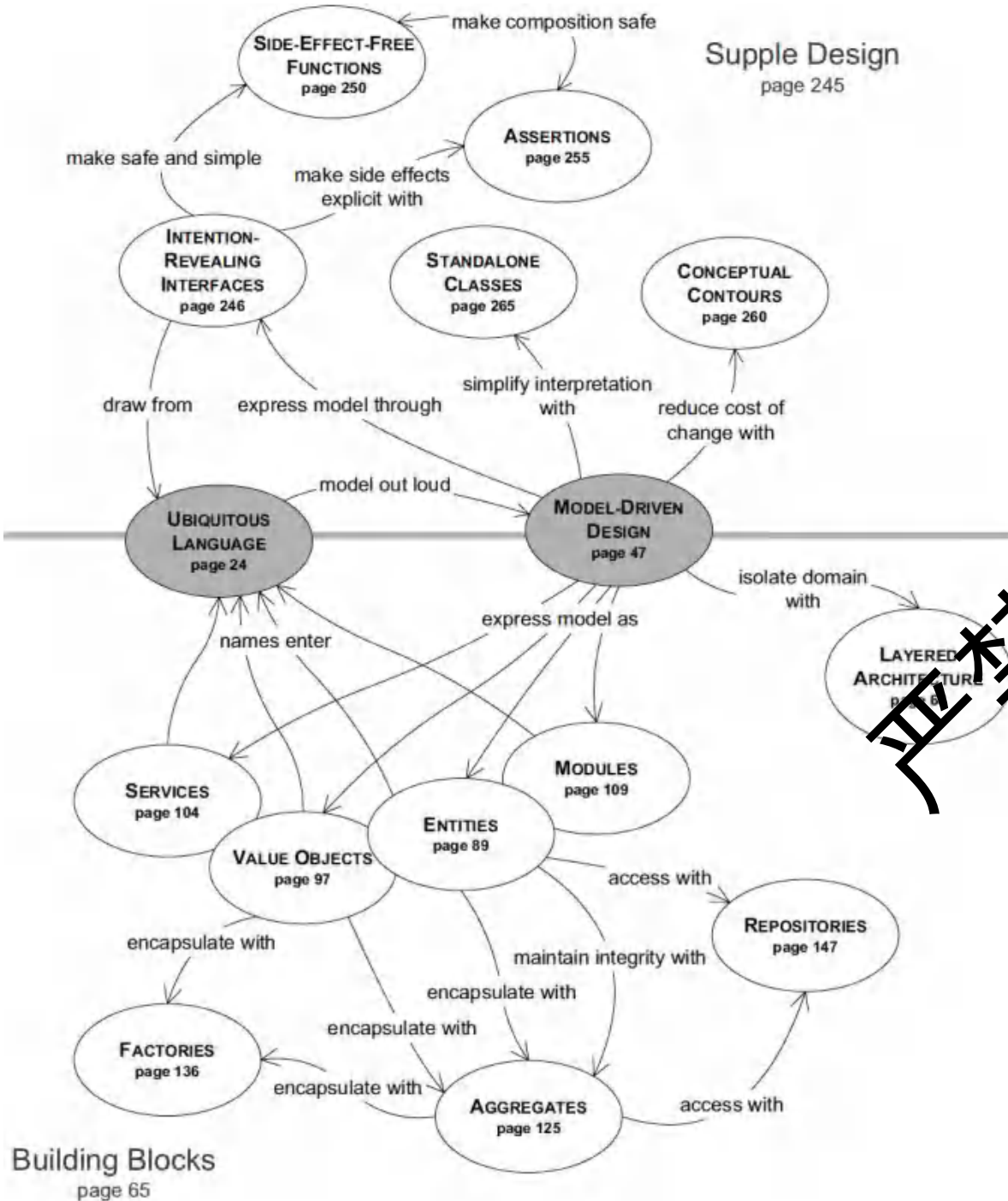
Pattern语言

1. 关于某个主题的完备的pattern目录;
2. 记录所有的可能性;
3. 通常描述各个pattern之间的关联关系: map

02



Example of pattern language Source: "SOA with REST" book



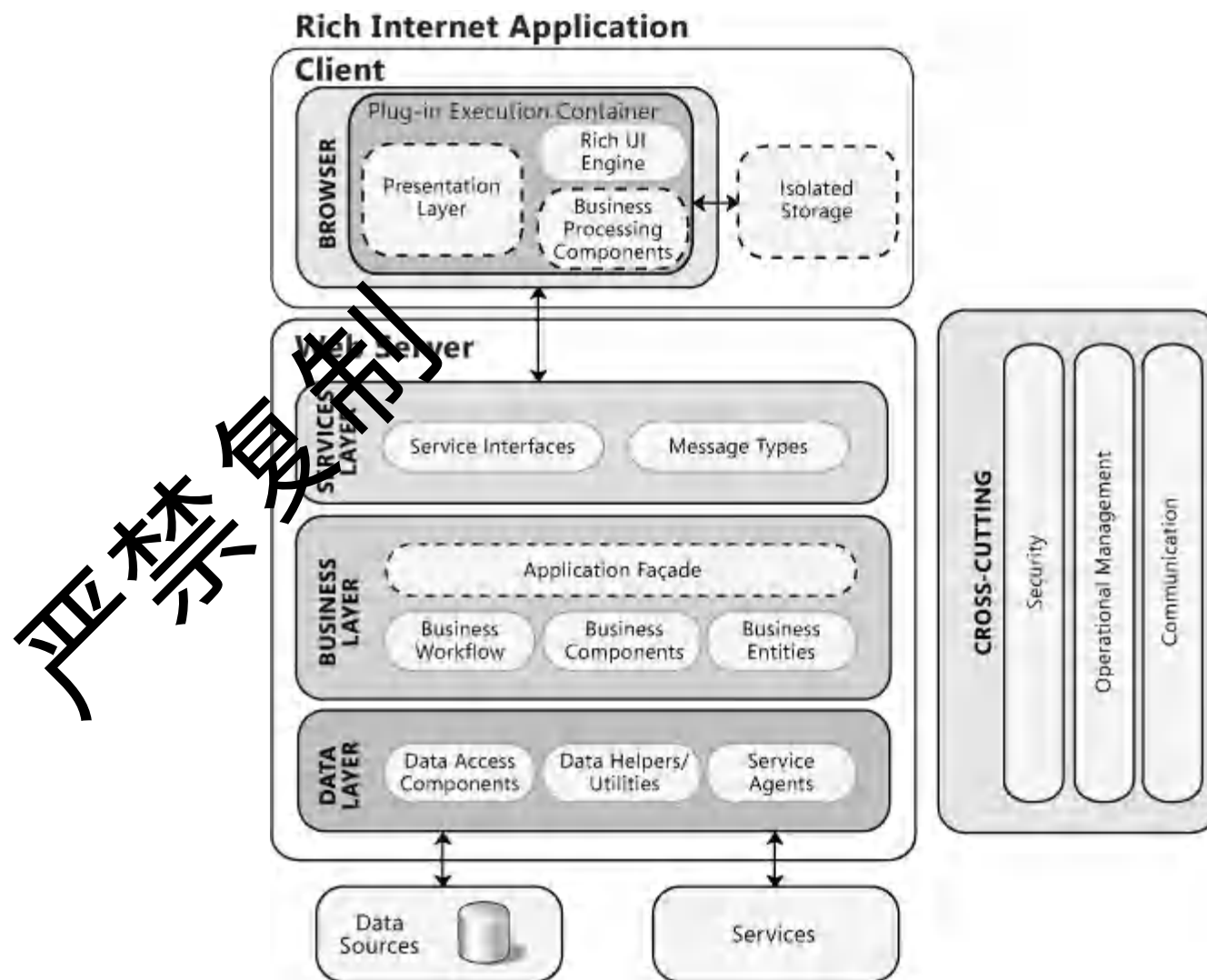
设计模式的catalog

Scope	Class	Purpose		
		Creational	Structural	Behavioral
		Factory Method (107)	Adapter (class) (139)	Interpreter (243) Template Method (325)
	Object	Abstract Factory (87) Builder (97) Prototype (117) Singleton (127)	Adapter (object) (139) Bridge (141) Composite (163) Decorator (175) Facade (185) Flyweight (195) Proxy (207)	Chain of Responsibility (223) Command (233) Iterator (257) Mediator (273) Memento (283) Observer (293) State (305) Strategy (315) Visitor (331)

Table 1.1: Design pattern space

参考架构

- 特定类型的应用的整体架构
 - 包含集中模式
 - 某个特定领域内的事实上的标准



[参考资料:](#)

模块化架构模式

1

layers分层模式

2

Aspect Oriented Programming (AOP)

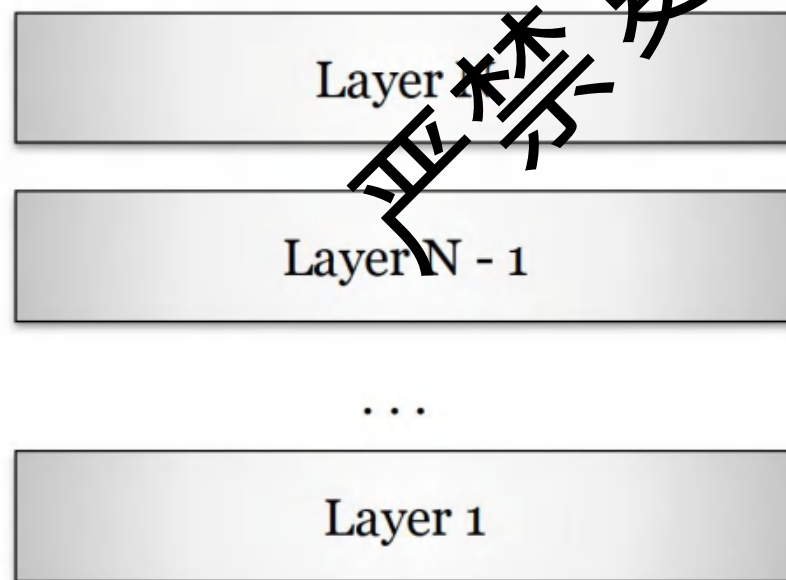
3

Domain Based (基于domain的划分方式)

严禁复制

分层模式

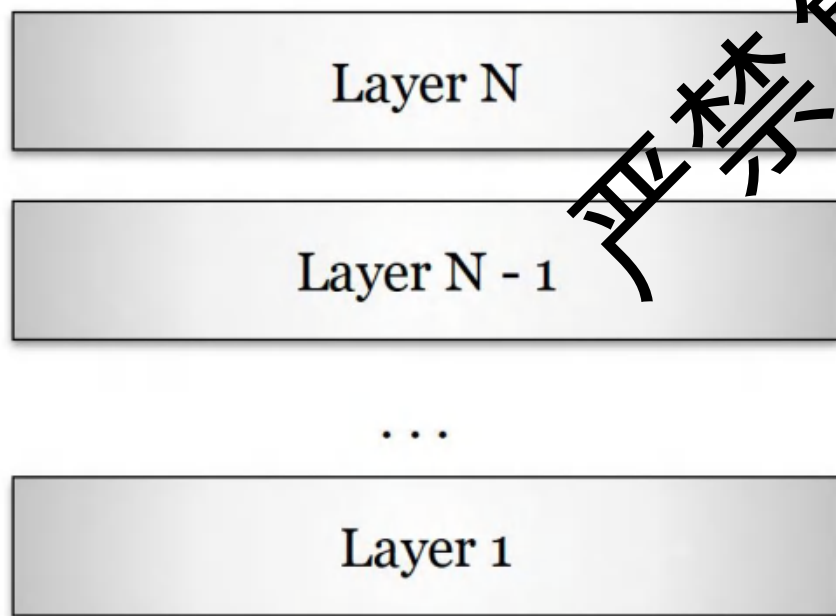
- 把软件模式分成不同的layer
- layer按照顺序排列
- 每个layer只暴露接口给上层的layer



分层模式

- 要素

- 层 (layer) : 在第N层通过接口暴露的功能集合;
- 层与层之间的顺序关系;



分层模式-约束

> 模块约束:

每个模块只属于一层

> 依赖关系约束:

可以作为client使用下层提供的服务
也可以作为server给上层提供服务

> 系统约束:

整个系统至少2层
不允许layer之间的循环调用

> 常见的变化:

严格: layer N只使用layer N-1的服务
宽松: Layer N使用Layer N-1.....1的所有服务

严禁复制

例子

- 以下是使用表格形式展示的图书管理系统架构设计，分为展示层、业务层、持久层和数据库层



Layer 不是 模块

区别:

一个模块可以分解成多个模块
(子模块)

一个layer不能分成多个layer

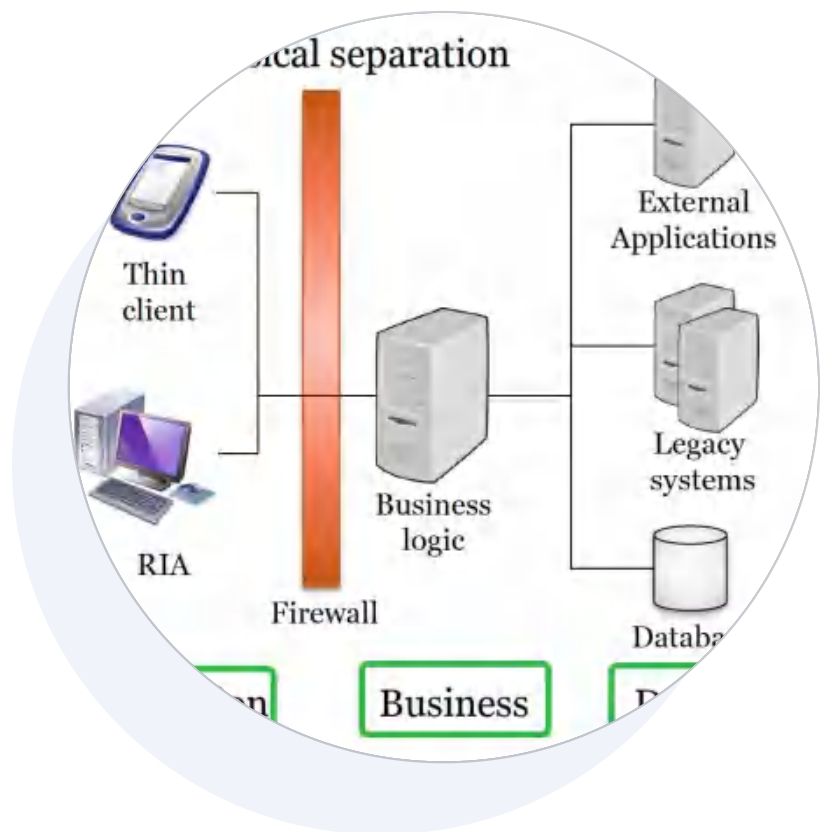
联系:

一个layer可以是一个模块

**一个layer可以包含多个模块,
layer不需要遵守SOLID原则**

用户管理
图书管理
日志管理
借书管理

Layer ≠ Tiers



Tiers:

Tiers 是指系统在物理部署上的分层。

每个 Tier 通常运行在不同的服务器上,通过网络通信。

Tiers 强调的是系统的物理分布和部署方式。

Layers:

Layers 是逻辑上的分层,表示代码或功能的组织结构。

Layers 可以运行在同一台机器上。

Tiers 例子——3层架构:

最常见的分布式架构之一。

展示层(Presentation Tier):负责用户界面和交互。

业务逻辑层(Application Tier / Logic Tier):处理核心业务逻辑。

数据层(Data Tier):负责数据存储和管理(如数据库)。

优点:职责分离,易于扩展和维护。

Layers的优势

01

不同layer分离不同级别的抽象概念

计算机概念是具体的：table, file;

业务概念是抽象的：transaction, order, process, shipment;

02

松耦合：各层可以独立发展

每层可以使用不同的实现，只需要保持接口不变

03

可复用：

改变一个layer只影响紧挨着的层
可以创建出标准接口作为共享库

04

容易测试：

只需要实现下层接口的mock即可

严禁复制

Layers的挑战

性能问题

每层都会产生性能消耗

可能导致单体应用

部署，可靠性，可扩展性

01

有时候很难实现

有些概念很难抽象到layer中

02

03

捷径

有时候需要跳过一些layers

04

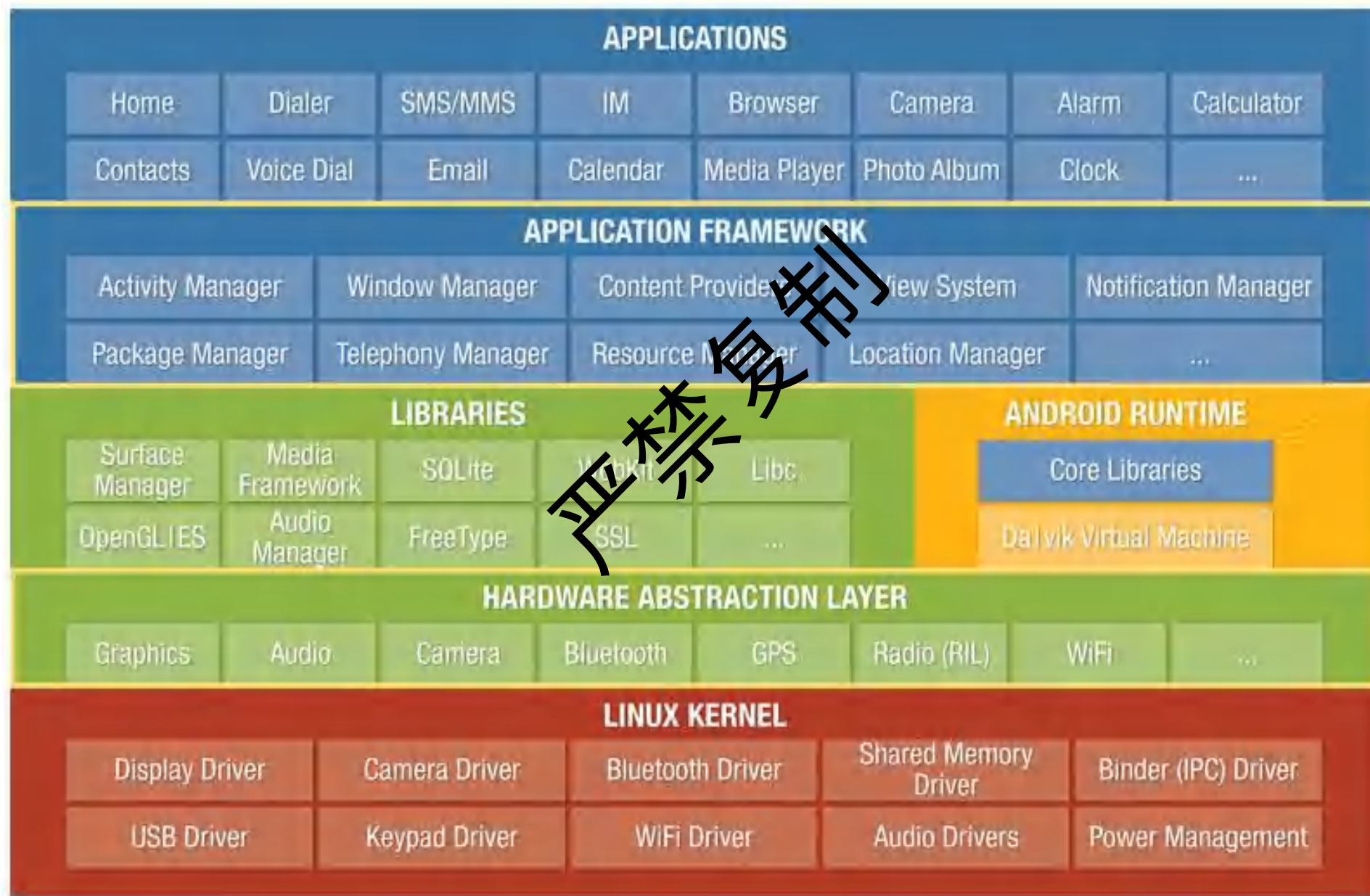
05

产生bad design: 透传

虽然什么也不做，但是layer中也需要进行处理

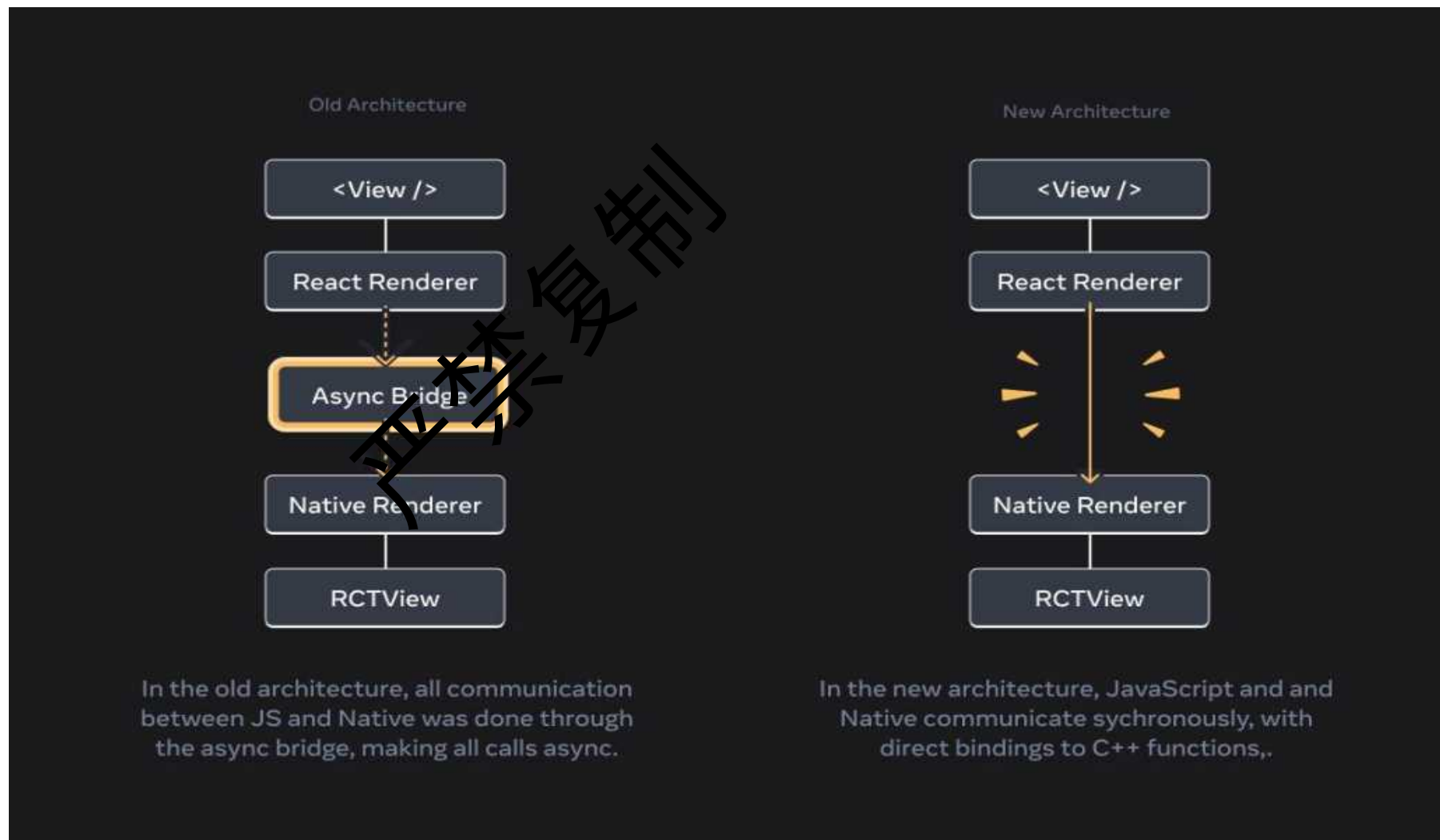
严禁复制

分层架构的例子：Android



分层架构的例子：React

- RCTView是iOS特有的视图组件



分层模式的特殊实现



VM虚拟机

3-Layers
(或多层)

虚拟机模式

虚拟机=不透明层

隐藏具体的OS实现

约束:

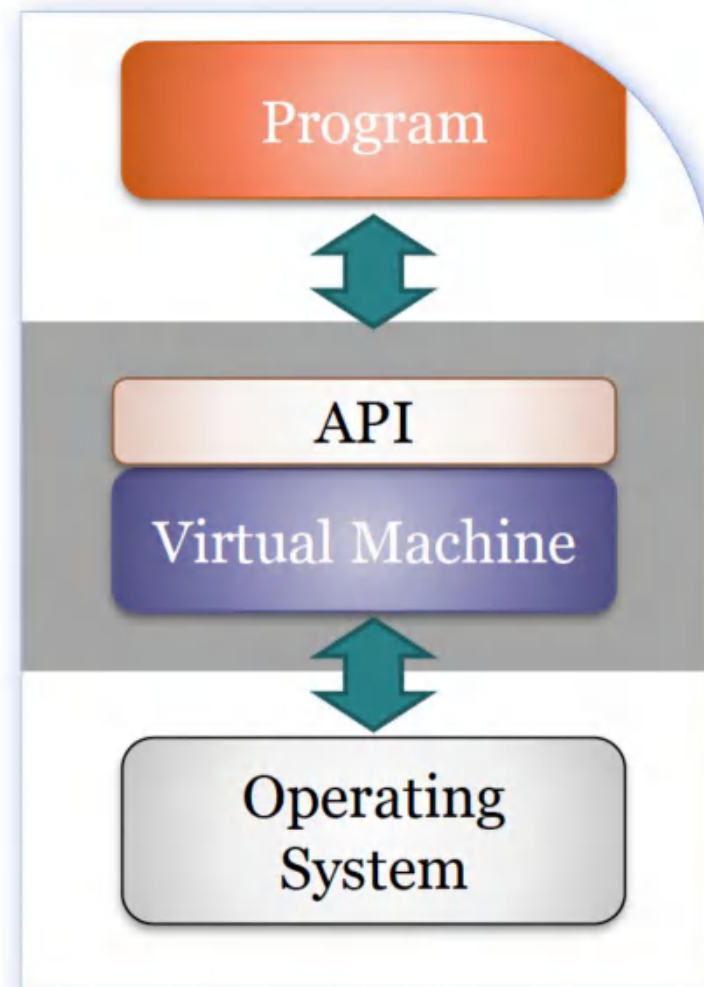
只能通过API访问操作系统

应用方式:

编程语言:JVM, CLR.net

Go,Python不使用VM

仿真软件:Android 仿真软件,VMware,
Docker不是VM



虚拟机-优势和挑战

• 优势

1. 可移植性
2. 简化软件开发
3. 使用高级语言开发
4. 简化仿真：
 1. 在虚拟机中可以创建多张网卡

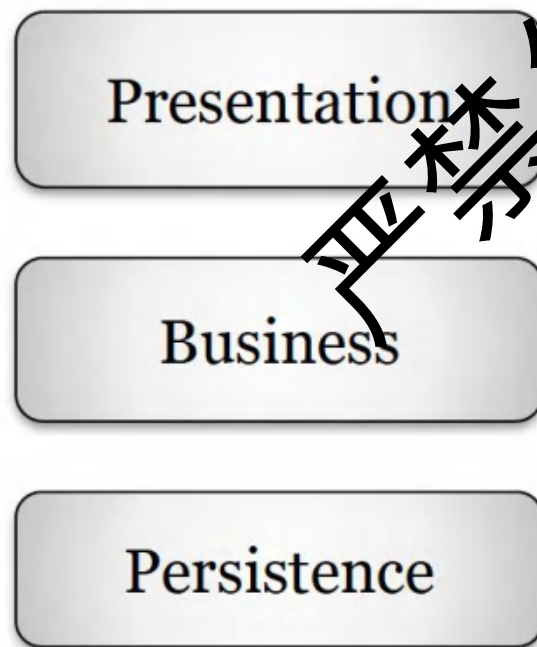
• 挑战

1. 性能 (bytecode翻译成机器代码)
2. JIT techniques (热点代码实时编译)
3. 使用虚拟机增加一个layer, 有计算损失

严禁复制

3-layers (N-layers)

- 技术分区
 - 每层都需要不同的技术能力



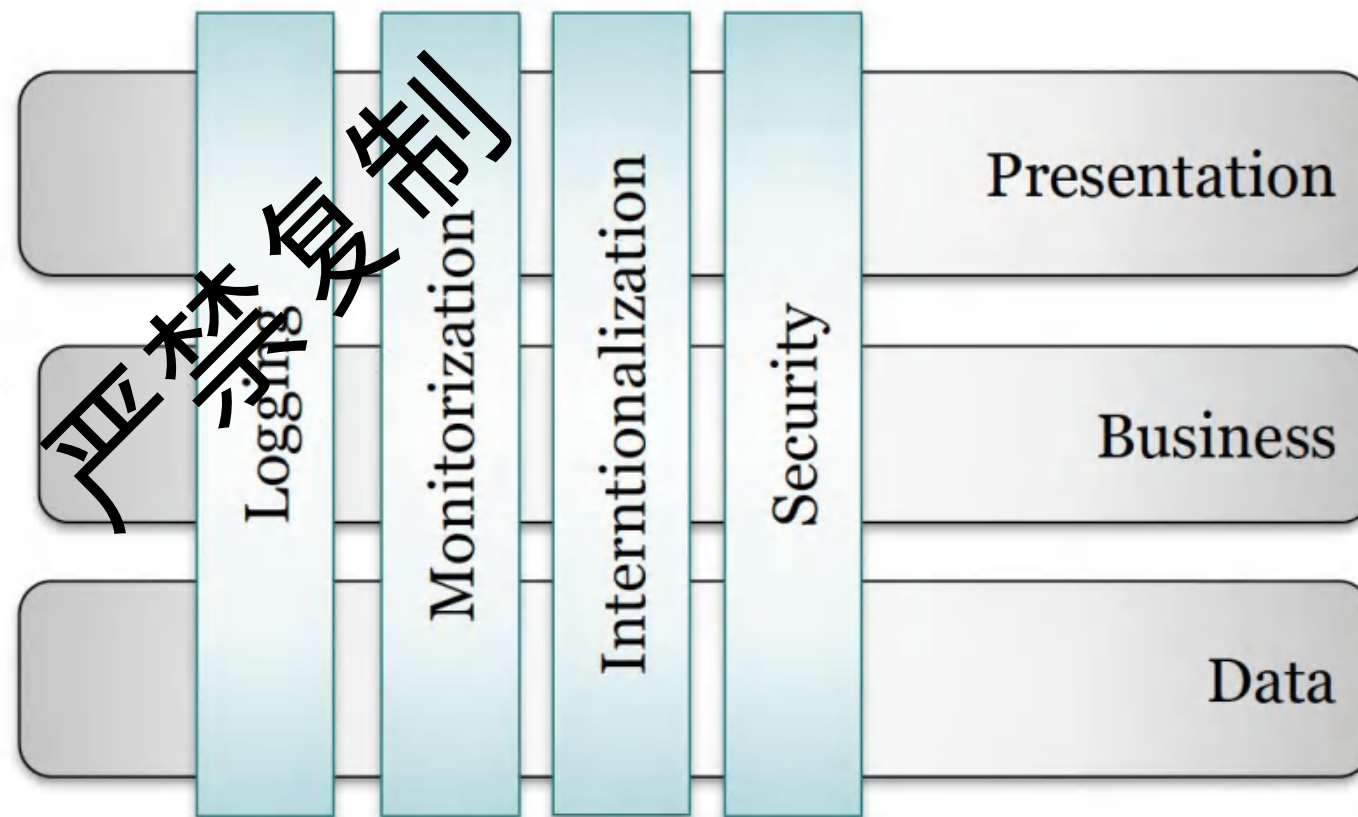
Aspect Oriented

严禁复制

Aspect Oriented

- Aspects

- 同时作用于多个layer的模块



切面

Aspect Oriented

- 要素
 - 切面关注点
 - 在一个应用系统中的多个地方都需要该功能
 - 例如：日志，监控，i18n，安全
 - 会产生复杂的代码
 - Aspect
 - 解决模块的切面关注点

严禁复制

Aspect Oriented

```
logger.info("开始转账, 从 " + fromAccount + " 到 " + toAccount + " 金额: " + amount);

boolean success = false;
Connection conn = null;
try {
    // 获取数据库连接
    conn = DatabaseUtil.getConnection();
    conn.setAutoCommit(false); // 开始事务

    // 执行业务逻辑
    AccountDao accountDao = new AccountDao(conn);
    accountDao.deduct(fromAccount, amount);
    accountDao.add(toAccount, amount);

    conn.commit(); // 提交事务
    success = true;

    logger.info("转账成功");
} catch (Exception e) {
    // 回滚事务
    try {
        if (conn != null) {
            conn.rollback();
        }
    } catch (Exception ex) {
        logger.error("回滚事务失败", ex);
    }

    logger.error("转账失败", e);
}
```

```
logger.info("开始存款, 账户: " + account + ", 金额: " + amount);

boolean success = false;
Connection conn = null;
try {
    // 获取数据库连接
    conn = DatabaseUtil.getConnection();
    conn.setAutoCommit(false); // 开始事务

    // 执行业务逻辑
    AccountDao accountDao = new AccountDao(conn);
    accountDao.add(account, amount);

    conn.commit(); // 提交事务
    success = true;

    logger.info("存款成功");
} catch (Exception e) {
    // 回滚事务
    try {
        if (conn != null) {
            conn.rollback();
        }
    } catch (Exception ex) {
        logger.error("回滚事务失败", ex);
    }

    logger.error("存款失败", e);
}
```

```
public boolean transfer(String fromAccount, String toAccount, double amount) {
    AccountDao accountDao = new AccountDao();
    accountDao.deduct(fromAccount, amount);
    accountDao.add(toAccount, amount);
    return true;
}

public boolean deposit(String account, double amount) {
    AccountDao accountDao = new AccountDao();
    accountDao.add(account, amount);
    return true;
}
```

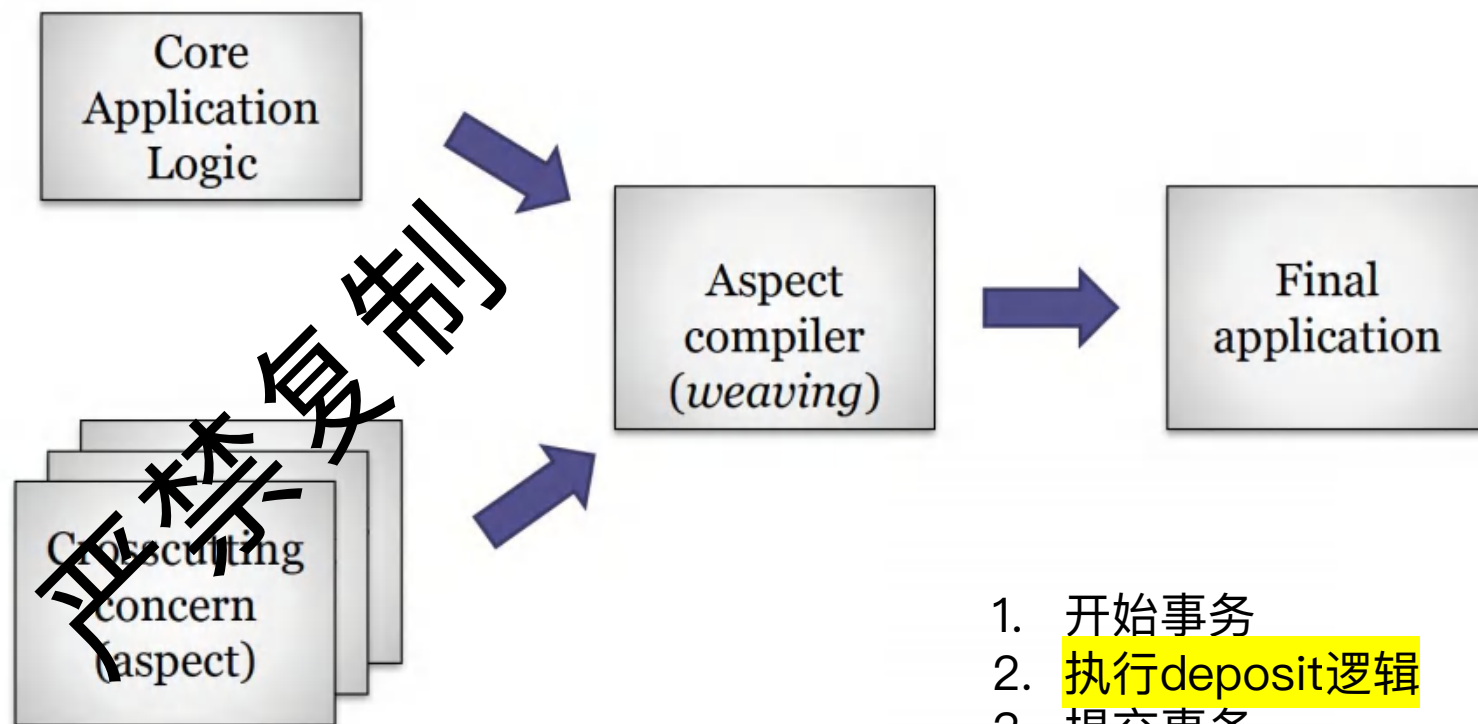
使用AOP

不使用AOP

AOP的结构

```
public class BankService {  
    private Logger logger = Logger.getLogger(BankService.class);  
  
    public void deposit(String account, double amount) {  
        logger.info("开始存款, 账户: " + account + ", 金额: " + amount);  
        try {  
            // 模拟存款逻辑  
            // 这里可能会抛出异常  
            logger.info("存款成功");  
        } catch (Exception e) {  
            logger.error("存款失败", e);  
            // 处理异常, 可能需要回滚  
        }  
    }  
}
```

```
@Aspect  
public class TransactionAspect {  
  
    @Around("execution(* com.example.bank.BankService.*(..))")  
    public Object manageTransaction(ProceedingJoinPoint joinPoint)  
        try {  
            // 开始事务  
            Object result = joinPoint.proceed(); // 执行目标方法  
            // 提交事务  
            return result;  
        } catch (Throwable e) {  
            // 回滚事务  
            throw e;  
        }  
    }  
}
```



1. 开始事务
2. 执行deposit逻辑
3. 提交事务
4. 如果发生异常
5. 回滚事务

AOP定义

- Join point:
 - 不体现在AOP的代码中，是编译器的一种机制；
 - 编译器在执行指令时，可以嵌入AOP的代码（如方法执行前，方法执行后）
- Aspect Contains:
 - Advice: 在选中的Join Point处执行的代码，包括执行时机（前、后、环绕等）
 - Pointcut: 定义哪些Join Point需要被处理，通过表达式筛选
 - 一个PointCut可以匹配多个Join Point

```
@Aspect
@Component
public class LoggingAspect {

    // 这是一个Pointcut, 定义了哪些Join Point会被选中
    // 这里选择了com.example.service包中所有类的所有方法
    @pointcut('execution(* com.example.service.*.*(..))')
    public void serviceMethods() {}

    // 这是一个Before Advice, 会在匹配的Join Point执行前运行
    @Before("serviceMethods()")
    public void logBefore(JoinPoint joinPoint) {
        System.out.println("Before executing: " + joinPoint)
    }

    // 这是一个After Returning Advice
    @AfterReturning(pointcut = "serviceMethods()", returning = "result")
    public void logAfterReturning(JoinPoint joinPoint, Object result) {
        System.out.println("Method " + joinPoint.getSignature() + " returned: " + result)
    }
}
```

Aspect Oriented的支持框架

- 核心AOP框架

- Spring AOP: Spring框架的一部分, 使用运行时代理实现AOP。轻量级, 易于使用, 但功能相对有限, 只支持方法执行连接点。
- AspectJ: 最完整、功能最强大的Java AOP解决方案。提供了编译期、编译后和加载时织入的能力, 支持几乎所有类型的连接点。

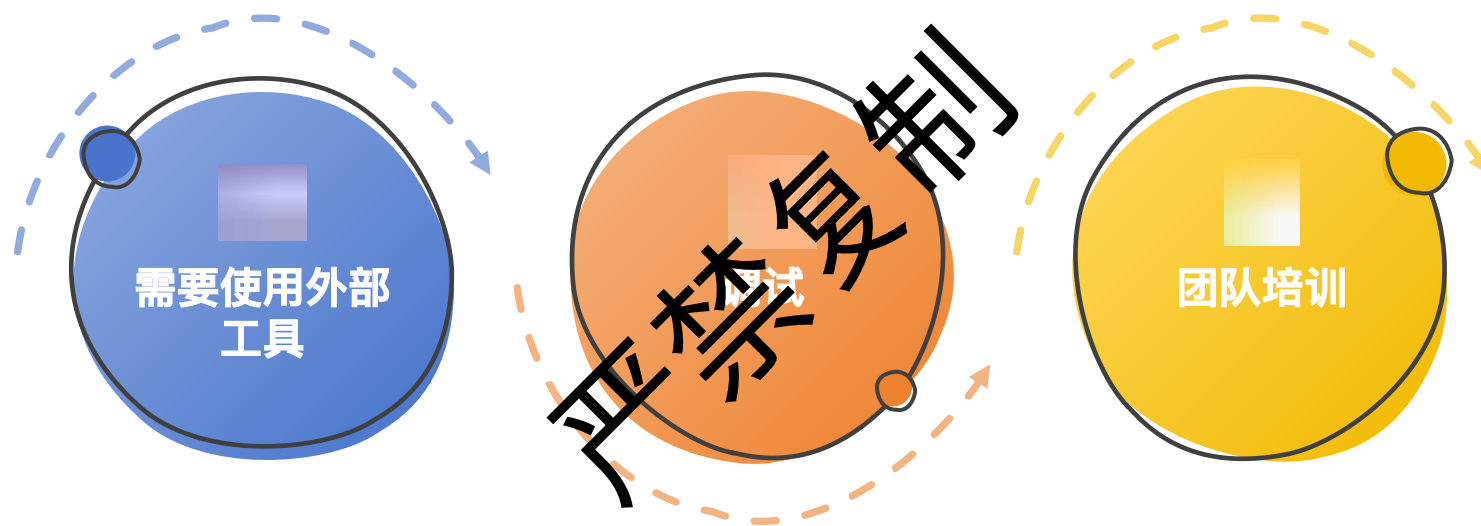
- 底层实现技术

- JDK动态代理: Java标准库的一部分, java.lang.reflect.Proxy, 但仅支持基于接口的代理。
- CGLIB: 代码生成库, 能够在运行时生成类的子类, 允许AOP应用于没有实现接口的类。
- Javassist: 一个字节码操作工具, 允许在Java字节码级别进行操作。
- ByteBuddy: 现代Java字节码操作和代码生成库, 用于创建和修改Java类。

- Java特性与API

- Java Instrumentation API: 允许在加载类时修改字节码。
- Java Agent: 与Instrumentation API结合, 可以在JVM启动时或运行时附加代理来转换类。
- 注解处理器: Java提供的编译时元数据处理机制。

AOP的挑战



Aspects compiler:
AspectJ
Spring

如果Aspect中有bug, 会影响其它模块 (在aspect中修改了参数)
代码流程更加复杂

学习新技能

AOP的变体——Data Context Interaction

- 传统OOP和DCI最本质的区别在于:
 - 传统OOP 将对象视为数据和行为的封装体，对象本身既包含状态也包含全部行为。这导致了对象责任可能过重，且业务流程分散在多个对象中。
 - DCI模式 将对象的"是什么"(数据)与"做什么"(行为)分离，引入了上下文和角色的概念。对象只在特定上下文中通过扮演角色获得行为，使得业务流程更加清晰可见。
 - Apache Polygene

```
// Account类中的转账方法(传统OOP)
public boolean transferTo(Account destination, double amount) {
    if (this.balance >= amount) {
        this.balance -= amount;
        destination.deposit(amount);
        return true;
    }
    return false;
}
```

```
// MoneyTransfer上下文类(DCI模式)
public void execute() {
    SourceAccount source = new SourceAccount(sourceAccount);
    DestinationAccount destination = new
        DestinationAccount(destinationAccount);

    if (source.hasSufficientBalance(amount)) {
        source.withdraw(amount);
        destination.deposit(amount);
    }
}
```

OOP的实现机制

// 传统OOP实现

```
class Account {  
    private String accountNumber;  
    private double balance;  
  
    public Account(String accountNumber, double init  
        this.accountNumber = accountNumber;  
        this.balance = initialBalance;  
    }  
  
    public String getAccountNumber() {  
        return accountNumber;  
    }  
  
    public double getBalance() {  
        return balance;  
    }  
}
```

```
public boolean transferTo(Account destination, double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("转账金额必须为正数");  
    }  
  
    if (this.balance >= amount) {  
        this.balance -= amount;  
        destination.balance += amount;  
        return true;  
    }  
    return false;  
}  
  
// 可能还会有其他方法如存款和取款  
public void deposit(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("存款金额必须为正数");  
    }  
    this.balance += amount;  
}  
  
public boolean withdraw(double amount) {  
    if (amount <= 0) {  
        throw new IllegalArgumentException("取款金额必须为正数");  
    }  
  
    if (this.balance >= amount) {  
        this.balance -= amount;  
        return true;  
    }  
    return false;  
}
```

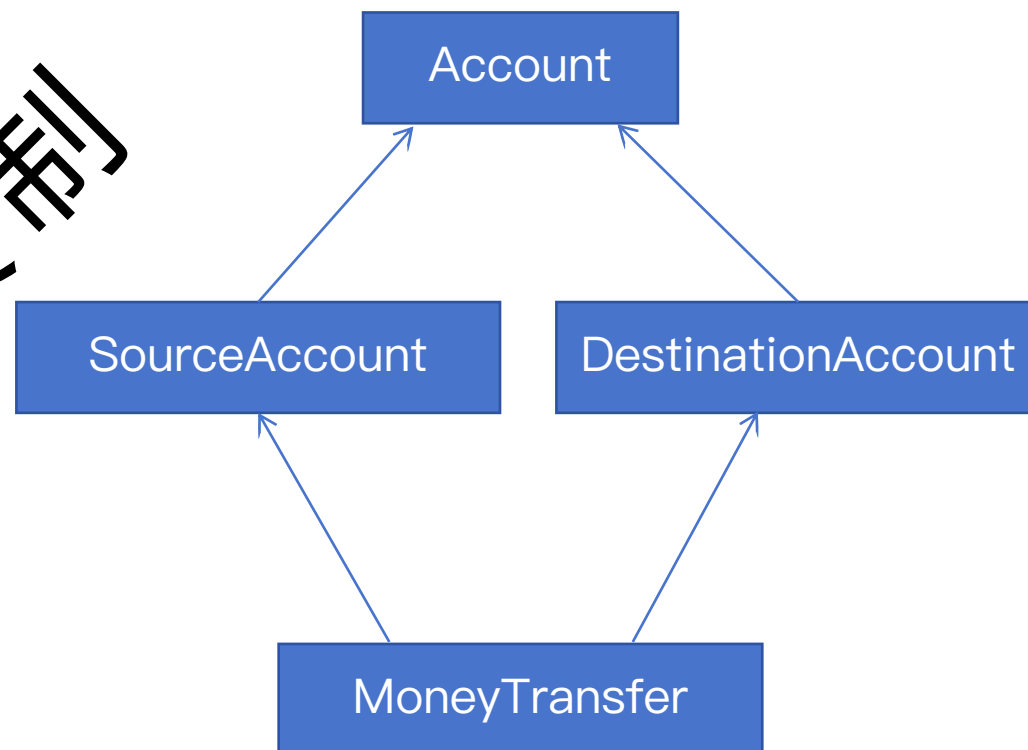
DCI的实现机制

```
// 上下文类
class MoneyTransfer {
    private Account sourceAccount;
    private Account destinationAccount;
    private double amount;

    // 构造函数初始化上下文
    public MoneyTransfer(Account source, Account destination,
        this.sourceAccount = source;
        this.destinationAccount = destination;
        this.amount = amount;
    }

    // 执行转账
    public void execute() {
        // 将账户对象与角色关联
        SourceAccount source = new SourceAccount(sourceAccount);
        DestinationAccount destination = new DestinationAccount(destinationAccount);

        // 角色间的交互
        if (source.hasSufficientBalance(amount)) {
            source.withdraw(amount);
            destination.deposit(amount);
        }
    }
}
```





04 复制 基于domain的模块划分

基于Domain的模块划分



领域驱动
设计



六边型架
构



数据集中
式

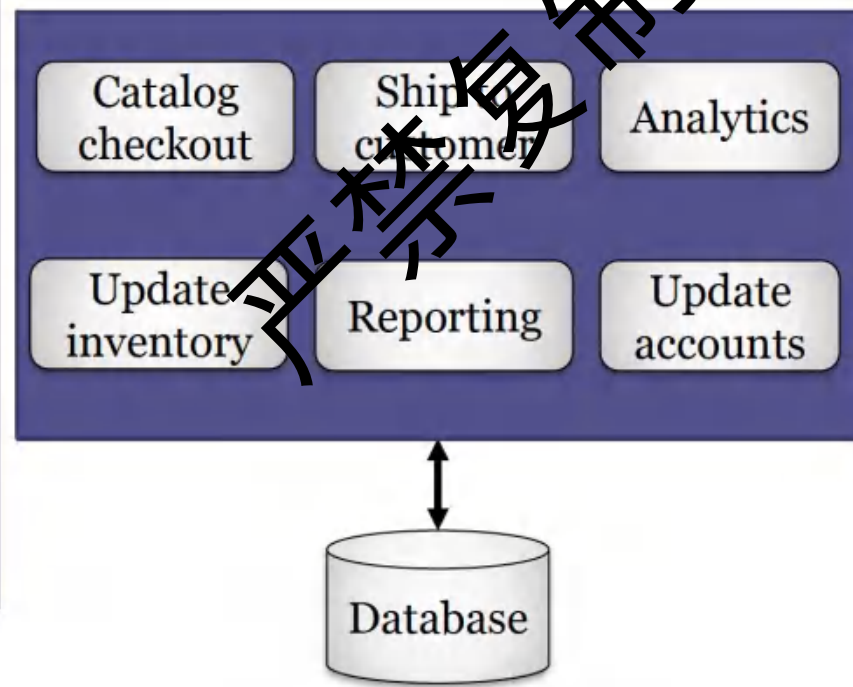
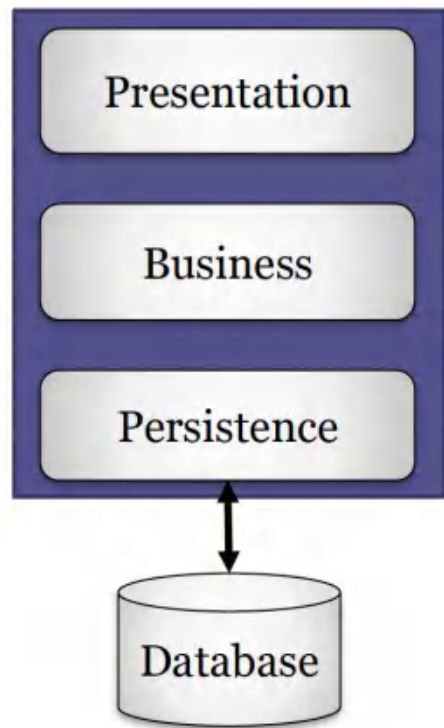


Naked
Objects

严禁复制

技术切分 vs 领域切分

- 传统的分层主要从技术角度考虑
- 基于领域的模块切分，首先考虑业务角度的切分，再考虑技术角度



数据模型 VS 领域模型



数据模型

物理形式

–数据表现形式: tables, columns, keys

逻辑形式

–数据结构: 实体, 关系



领域模型

领域内的概念模型

限界上下文, 领域词汇, 形成通用语言

领域行为: 业务规则

基于领域的模块划分

- DDD–领域驱动设计
- 六边型架构
- 数据中心
- 分层的DDD
- Naked Objects

严禁复制

DDD

01 软件开发的通用方法论

02 Eric Evans 2004年提出

03 将系统实现与不断发展变化的领域联系起来

04 实现技术专家和领域专家的合作

05 统一语言：领域专家和研发团队共用的词汇

严禁复制

DDD-要素

- Bounded context限界上下文
 - 描述领域的边界，在BC之中，一个名词只有一个含义
- entity实体对象
 - 根据ID来识别的对象，例如people
- Value Object值对象
 - 根据值来识别的对象，例如工期
- Aggregate聚合根
 - 一组对象工作协作实现一些业务过程，例如订单结算
- Repository持久化服务
- Factory创建对象的服务，创建entity, value object, aggregate
- Services封装复杂的算法，访问其它领域对象
- domain event实现跨领域的沟通协作

禁止复制

DDD要素-限界上下文

- 在 DDD（领域驱动设计）中，限界上下文（Bounded Context）是用于划分业务领域的边界，确保每个上下文中的术语和概念具有明确的含义，避免歧义。以下以 订单 为例，说明在 购买领域、结算领域 和 物流领域 中，限界上下文如何避免歧义。

```
public class Order {  
    private String orderId;  
    private List<OrderItem> items;  
    private double totalAmount;  
  
    public void addItem(OrderItem item) {  
        items.add(item);  
        totalAmount += item.getPrice() *  
    }  
  
    public void checkout() {  
        // 检查库存并生成订单  
    }  
}
```

```
public class PaymentOrder {  
    private String orderId;  
    private double totalAmount;  
    private PaymentStatus status;  
  
    public void processPayment(Payment  
        // 处理支付逻辑  
        status = PaymentStatus.PAID;  
    }  
}
```

```
public class ShippingOrder {  
    private String orderId;  
    private Address shippingAddress;  
    private ShippingStatus status;  
  
    public void ship() {  
        // 生成配送单并更新状态  
        status = ShippingStatus.SHIPPED;  
    }  
}
```

DDD要素-聚合根

- 一致性：通过聚合根 Order 控制对 OrderItem 的修改，确保订单的总金额始终正确。
- 封装性：外部只能通过聚合根 Order 来操作订单，无法直接修改 OrderItem 或 Address。
- 边界：Order 是聚合的入口点，定义了聚合的边界。

```
public class Order {  
    private final String orderId;  
    private final String customerId;  
    private final Address shippingAddress;  
    private final List<OrderItem> items;  
    private double totalAmount;  
}
```

```
public void addItem(OrderItem item) {  
    items.add(item);  
    totalAmount += item.calculateTotal();  
}
```

```
public void removeItem(String productId) {  
    items.removeIf(item -> item.getProductId().equals(productId));  
    recalculateTotal();  
}
```

```
private void recalculateTotal() {  
    totalAmount = items.stream()  
        .mapToDouble(OrderItem::calculateTotal)  
        .sum();  
}
```

DDD-约束



聚合根内的实体只能通过
聚合根的方法访问

需要保持聚合根的一致性



所有存储操作必须由
Repository完成

根据需要可酌情增减文字，以便
观者准确地理解您传达的思想



Value Objects是只读的

只能新增，不能修改

领域驱动的优势

简化团队的沟通交流

研发团队（多个），产品团队，QA团队，业务团队，
使用同一种语言进行沟通；
跨团队沟通交流耗时且容易导致错误

增强数据质量和一致性

消除了数据的歧义，不同模块使用同样的语言

反应了领域的内部结构

能够处理领域的变化
可以共享和复用领域模型

提升系统的可测试性

可以构造一个mock领域来进行测试（例如为了测试
订单域，可以mock支付域）

严禁复制

领域驱动的挑战

01

和领域专家合作

没有领域专家
领域专家不愿意花时间交流
领域专家不能清晰的陈述

禁止复制

分析阶段陷于停滞

领域知识过于丰富，难以建模



03

对研发的约束

需要研发人员使用统一词汇，使用
正确的对象类型

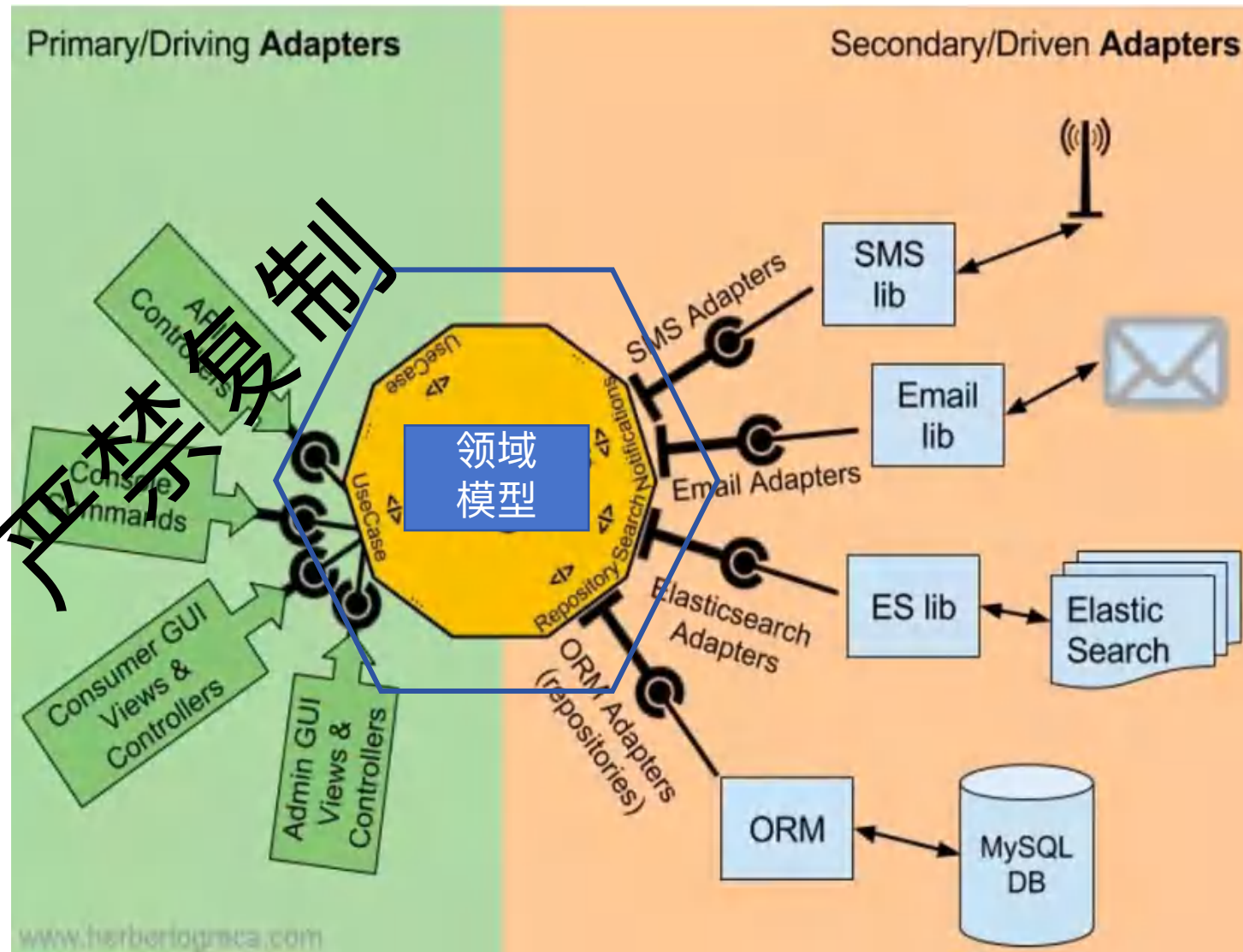
六边型架构模式

- 其它名字

- 端口和适配器模式
- 洋葱模型
- clean架构

- 基于领域模型

- 基础设施和基础框架在外层
(传统的分层架构不同)
- 所有访问都是基于Port和Adapter



六边型架构的要素

领域模型

01

业务逻辑：实体和关系
普通的java对象



Ports

02

对外的服务接口（需要别人实现，不同于模块接口）

Adapters

03

一个适配器对应一个外部元素

例如：REST，DB，ES，
Logging，Cache

六边型架构的优势

容易理解

核心清晰，与辅助性业务
分离开

适应性

能够快速改变以适应
市场变化

可测试性

替换某个基础服务比较容易

稳定性

不随技术和框架的发展而
变化

六边型架构的挑战

> 将领域逻辑和存储模型分开

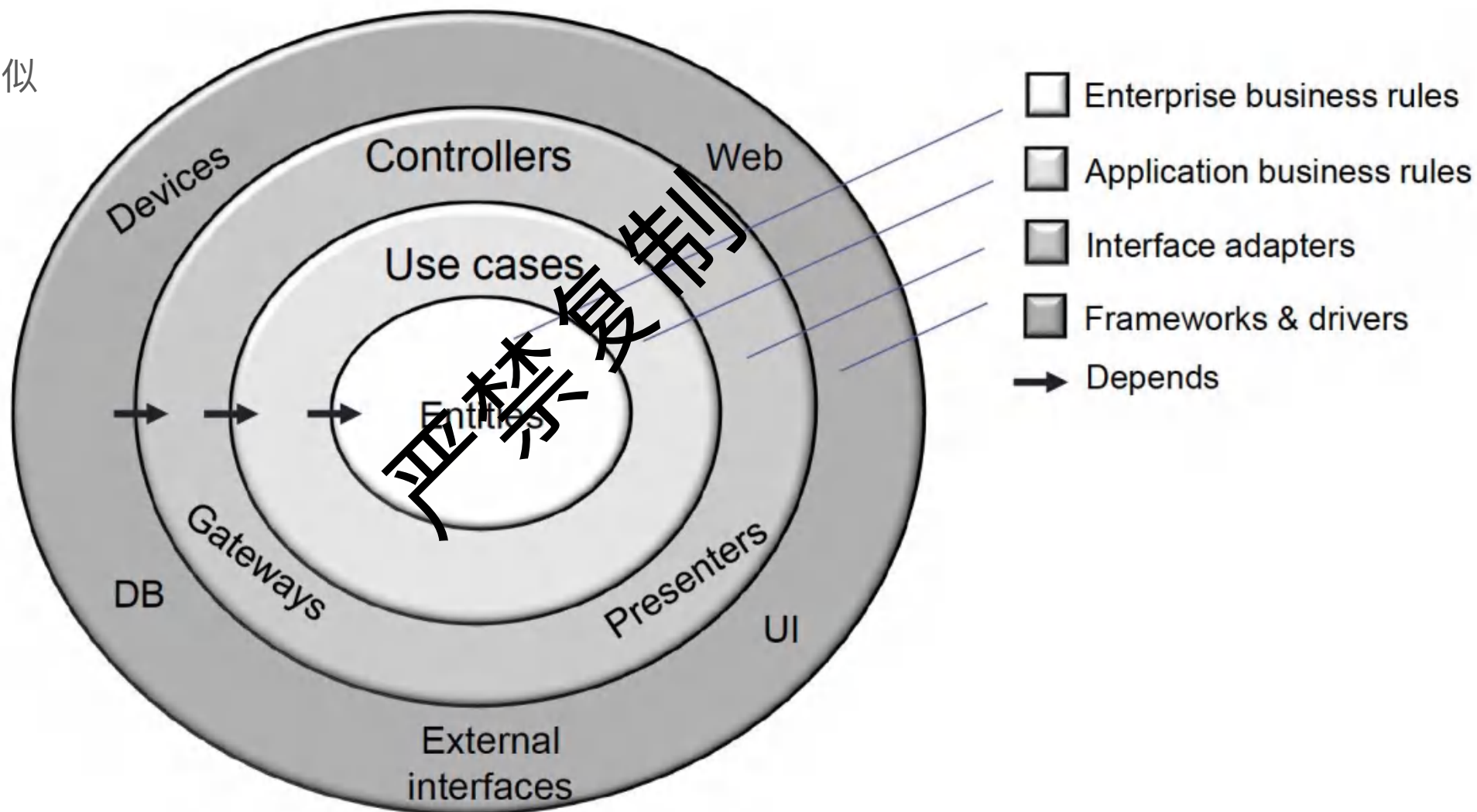
大多数ORMapping模型中绑定了业务逻辑和存储引擎

Ports和adapter的不对称性

有的ports容易实现
有的ports难以实现

Clean架构

- 与六边型架构类似



以数据为中心的领域划分

- 怎么划分领域？
- 以数据对象为基础实现领域划分
 - 每个领域维护一个核心对象的CRUD
 - 基于CRUD实现业务流程
- 优势
 - 可以实现半自动化的代码生成
 - 可以快速实现
- 挑战
 - 难以实现更复杂的 领域
 - 陷于贫血模型：class只有getter和setter，通过procedure来把对象串联起来

```
public boolean executeCheckout(Order order, Payment payment) {  
    // 1. 验证订单  
    if (!validateOrder(order)) {  
        System.out.println("订单验证失败");  
        return false;  
    }  
  
    // 2. 计算总额 (包括税费、运费等)  
    double finalAmount = calculateFinalAmount(order);  
    order.setTotalAmount(finalAmount);  
  
    // 3. 处理支付  
    boolean paymentSuccess = processPayment(order, payment);  
    if (!paymentSuccess) {  
        System.out.println("支付处理失败");  
        order.setStatus("PAYMENT_FAILED");  
        return false;  
    }  
  
    // 4. 更新订单状态  
    order.setStatus("PAID");  
    order.setPaymentDate(new java.util.Date());  
  
    // 5. 更新产品库存  
    updateInventory(order);  
  
    // 6. 生成收据  
    String receipt = generateReceipt(order, payment);  
    order.setReceipt(receipt);  
  
    // 7. 通知客户
```

以能力为中心的领域划分

- 围绕这一个能力来划分领域

- 结算能力
- 物流规划能力
- 预测能力

- 优势

- 更容易聚焦
- 纵深发展

- 挑战

- 可能出现重叠
- 比较抽象，初学者难以掌握

1. 课程管理能力：专注于课程的生命周期管理。
2. 用户学习领域：专注于用户的学习行为和学习进度管理
3. 支付结算领域：专注于支付和结算流程
4. 内容创作领域：专注于课程内容的创作和管理
5. 用户管理领域：专注于用户的生命周期管理。

严禁复制

纯对象模式

- 在领域对象中加入业务逻辑
- 利用框架基于领域对象生成：
 - UI
 - service
 - database
- 框架
 - Apache Causeway
 - low code

```
// 领域对象定义
@DomainObject(
    nature = Nature.ENTITY,
    objectType = "Customer"
)
```

```
public class Customer {
```

```
    // 属性
```

```
    @Property(
        editing = Editing.ENABLED
    )
```

```
    @PropertyLayout(
        named = "姓名",
        describedAs = "客户的全名"
    )
```

```
    @Getter @Setter
    private String name;
```

```
    @Property(
        editing = Editing.ENABLED
    )
```

```
    @PropertyLayout(
        named = "邮箱",
        describedAs = "客户联系邮箱"
    )
```

```
    @Getter @Setter
    private String email;
```

```
    // 关联
```

```
    @Collection
    @CollectionLayout(
        named = "订单"
    )
```

```
    @Getter @Setter
    private List<Order> orders = new ArrayList<>();
```

```
    // 行为方法
```

```
    @Action(
        semantics = SemanticsOf.NON_IDEMPOTENT
    )
```

```
    @ActionLayout(
        named = "下单",
        describedAs = "为该客户创建新订单"
    )
```

```
    public Order placeOrder(
        @Parameter(optional = Optionality.MANDATORY)
        @ParameterLayout(named = "订单日期")
        final LocalDate orderDate) {
```

```
        Order order = new Order();
        order.setCustomer(this);
        order.setOrderDate(orderDate);
        order.setStatus("新订单");
        orders.add(order);
        return order;
    }
```

纯对象模式

优势

适配领域
容易维护（甚至不需要改
代码）

挑战

不支持个性化定制

应用

Naked Objects (.net)
Apache Isis(Java)

THANK YOU

汇报人：北京交通大学软件学院

严禁复制

